

Day 2 — Signed Bits

Curriculum alignment: Stage 1 — Foundations of Computation · Signed Numbers and Bitwise Reasoning

Target time: ~3 hours (≈ 75 minutes concept work, ≈ 90 minutes labs, ≈ 15 minutes mastery explanation)

A byte arrives from a device:

```
text 11010110
```

Is it 214? Is it -42 ? Is it a collection of status lights? Did an arithmetic result overflow?

The pattern alone cannot answer. We need the rule that gives the pattern meaning.

The previous foundation established the physical-to-logical path: a voltage falls into an agreed range, circuitry classifies it as one of two states, and we write those states as 0 and 1. Today we give groups of those states useful structure.

We will follow one engineering problem from beginning to end:

```
finite bit patterns
  → signed-number encoding
  → Boolean operations on selected bits
  → packed device fields
  → fixed-width arithmetic and flags
  → the beginnings of an ALU
```

The important habit is not merely converting binary quickly. It is asking:

What interpretation rule is active, what width applies, and what facts were lost when the hardware kept only that width?

1. Why machines need a representation for negative values

People can write a minus sign beside a magnitude. A machine cannot get that extra symbol for free. Every stored distinction needs a physical state; every arithmetic rule needs circuitry or a sequence of operations.

This problem was visible before electronic computers. A mechanical calculator had only a fixed number of wheel positions. When a wheel passed its last position, it returned to its first and carried into the next wheel. Designers learned that complement arithmetic could turn subtraction into addition followed by discarding a carry. That mattered because one reliable adding mechanism was easier to build than unrelated machinery for every combination of signs.

Binary registers have the same circular constraint:

```
decimal wheel:  ... 8 → 9 → 0 → 1 ...  
8-bit register: ... 254 → 255 → 0 → 1 ...
```

An eight-bit register has exactly 256 physical patterns:

```
00000000 through 11111111
```

Unsigned binary assigns them to 0 through 255. But temperatures fall below zero, account balances can be negative, offsets can point backward, and subtraction can produce a negative result. We need a different mapping of those same patterns.

The design question is therefore:

Which mapping gives us negative values while making arithmetic hardware as simple and regular as possible?

First attempt: signed magnitude

The most human-looking answer is to use the highest bit as a sign and the remaining bits as the magnitude:

```
4-bit signed magnitude  
  
0011 → +3  
1011 → -3
```

But this produces two zeros:

```
0000 → +0  
1000 → -0
```

It also complicates arithmetic. Before adding, the machine must inspect both signs and decide whether to add magnitudes, subtract them, and which sign to attach. The notation is easy for a person to glance at, but the datapath needs extra cases.

Signed magnitude still exists where its trade-offs make sense, including the sign-and-significand view of common floating-point formats. It was simply a poor universal answer for ordinary integer arithmetic.

Second attempt: one's complement

One's complement represents a negative value by inverting every bit of the positive value:

```
0011 → +3  
1100 → -3
```

This moves closer to complement arithmetic, but it still has two zeros:

```
0000 → +0  
1111 → -0
```

Addition also requires an end-around-carry correction: a carry leaving the highest position must be added back into the low position. That is workable, but it leaves an irregular rule in a very common operation.

The extra one that regularizes the circle

Two's complement forms $-x$ by inverting the bits of x and adding one:

```
+3      0011
invert   1100
add one  + 0001
-----
-3      1101
```

The added one is not a ritual to memorize. It removes negative zero and makes the encoding line up with fixed-width modular arithmetic.

2. Two's complement as arithmetic modulo 2^n

With four bits, hardware retains only the low four result bits. There are $2^4 = 16$ positions, so arithmetic circles modulo 16.

pattern	unsigned reading	signed two's-complement reading
-----	-----	-----
0000	0	0
0001	1	1
...		
0111	7	7
1000	8	-8
1001	9	-7
...		
1111	15	-1

To encode -3 , choose the non-negative representative of the same residue:

$$-3 \bmod 16 = 16 - 3 = 13 = 1101_2$$

This gives two equivalent ways to decode an n -bit pattern whose high bit is 1:

$$\text{signed value} = \text{unsigned value} - 2^n$$

or:

invert $\rightarrow$ add one $\rightarrow$ read magnitude $\rightarrow$ attach minus sign

Try the running byte:

11010110

Unsigned:

$$128 + 64 + 16 + 4 + 2 = 214$$

Signed, using the modular rule:

$$214 - 256 = -42$$

Signed, using invert-plus-one:

```
11010110
invert → 00101001
add 1 → 00101010 = 42
therefore the original pattern represents -42
```

The two methods agree because they are two views of the same mathematics.

Why one binary adder can serve both interpretations

Consider four-bit $5 + (-3)$:

```
  0101
+ 1101
-----
1 0010
```

The four-bit destination retains `0010`, which is 2. The carry leaving the width is discarded. No separate negative-number adder was needed.

The gates in an adder do not label one input “signed” and another “unsigned.” They combine bit positions and generate sums and carries. Signedness tells us how to interpret the operands, result, and status flags.

That regularity is the central engineering reason two's complement became the normal integer representation:

- one representation for zero;
- addition and subtraction can share an adder;
- negation is invert-plus-one;
- the high bit directly reveals the sign under the signed interpretation;
- widening can preserve a signed value by copying that high bit.

The range and its asymmetry

For n bits:

```
unsigned:      0 through  $2^n - 1$ 
two's complement:  $-2^{n-1}$  through  $2^{n-1} - 1$ 
```

For eight bits:

```
unsigned:      0 through 255
two's complement: -128 through 127
```

There are 128 negative values, one zero, and 127 positive values. The most negative pattern, `10000000`, has no positive counterpart in the same width. That is why operations such as taking the absolute value of the most negative signed integer require special care.

Quick derivation, not recognition

Without a calculator, decode `11100101`:

1. Read it as an unsigned sum of place values.
2. Subtract 256 for the signed reading.
3. Verify by invert-plus-one.

If you can derive both readings, you understand the representation. Merely remembering that “a leading 1 means negative” is not enough; that statement is true only after selecting a signed two's-complement interpretation.

3. When a word is a control panel, not a number

Now reinterpret our running pattern as a fictional device register:

bit:	7	6	5	4	3	2	1	0
meaning:	READY	ERROR	MODE[2:0]			RSV	IRQ_EN	START

Suppose the device specification says:

bit 7	READY	device has completed an operation
bit 6	ERROR	operation failed
bits 5:3	MODE	operating mode, values 0–7
bit 2	reserved	preserve or write zero as specified
bit 1	IRQ_EN	interrupt generation enabled
bit 0	START	request a new operation

The byte is still physically `11010110`, but adding its place values is now usually the wrong question. Software needs to inspect or modify selected positions while preserving the others.

The tools come from Boolean algebra. Long before transistor CPUs, logical algebra described relationships such as AND, OR, and NOT. Communications engineers later had large networks of electromechanical relays: current in a coil moved a contact, and contacts in series or parallel naturally behaved like logical conditions. Work connecting Boolean algebra to switching networks showed that an expression on paper could systematically become wiring.

Relay adders then made the bridge concrete:

```
logical relation
  → arrangement of switches
  → physical output state
  → calculation
```

Modern CPUs replace moving relay contacts with transistor networks, but the logic remains recognizable. A C expression such as:

```
reg & ERROR_MASK
```

is an abstract request that ultimately becomes controlled state transitions through logic gates.

4. Four bitwise operations, four mask intentions

AND: keep selected information

AND produces 1 only if both input bits are 1:

```
0 & 0 = 0    0 & 1 = 0
1 & 0 = 0    1 & 1 = 1
```

Test the ERROR bit, bit 6:

```
register: 11010110
mask:     01000000
          ----- &
result:   01000000
```

The nonzero result says bit 6 was set.

```
if (reg & (1u << 6))
    puts("ERROR is set");
```

AND can also force selected positions to zero. To clear bit 1:

```
reg &= ~(1u << 1);
```

The inner shift makes `00000010`; NOT makes a mask with zero only at the target position; AND preserves every other bit.

OR: force selected bits on

OR produces 1 if either input is 1:

```
0 | 0 = 0    0 | 1 = 1
1 | 0 = 1    1 | 1 = 1
```

Set `IRQ_EN`:

```
reg |= 1u << 1;
```

A 1 in the mask forces the target on. A 0 in the mask preserves the original bit.

XOR: toggle selected bits

XOR produces 1 when its inputs differ:

```
0 ^ 0 = 0    0 ^ 1 = 1
1 ^ 0 = 1    1 ^ 1 = 0
```

Toggle `START`:

```
reg ^= 1u << 0;
```

A 1 in the mask flips a bit; a 0 preserves it. Applying the same XOR mask twice restores the original:

```
x ^ mask ^ mask = x
```

That reversible property helps explain XOR's appearance in checksums, difference tracking, and cryptographic constructions. Those uses add further rules; XOR by itself is not encryption.

NOT: invert the active width

NOT flips every bit:

```
~00110110 = 11001001    if the active width is exactly eight bits
```

The width qualification matters in C. Small integer operands normally undergo integer promotion before `~`:

```
uint8_t x = 0x36;
uint8_t inverted = (uint8_t)~x;
```

The operation commonly occurs at `int` width, and the cast narrows the result back to eight bits. This is our first warning that a variable's storage width does not necessarily determine an expression's evaluation width.

5. Shifts construct and position fields

Shifts move bit positions:

```
00000001 << 3 = 00001000
00110000 >> 4 = 00000011
```

For unsigned values, a representable left shift by one corresponds to multiplication by two; right shift by one corresponds to integer division by two. In systems code, their more important role is often positional.

Our MODE field occupies bits 5:3. Its mask is:

```
00111000 = 0x38
```

Extract it in two steps:

```
unsigned mode = (reg & 0x38u) >> 3;
```

```
register: 11010110
mask:    00111000
AND:     00010000
shift:   00000010 → MODE is 2
```

The general shape is:

```
field = (word & FIELD_MASK) >> FIELD_SHIFT;
```

Replacing a field also takes two deliberate steps:

```
reg &= ~MODE_MASK;           /* clear old field */
reg |= (new_mode << MODE_SHIFT) & MODE_MASK;
```

Masking the shifted value prevents an oversized input from leaking into neighboring fields. Validating the input may still be appropriate.

Do not use this:

```
reg |= new_mode << MODE_SHIFT;
```

OR can set field bits but cannot clear old ones. If the old MODE is `111` and the new MODE is `010`, blind OR leaves `111`.

Reserved and side-effect bits

“Reserved” does not mean “available to software.” A device specification may require such bits to be written as zero, preserved during read-modify-write, or ignored when read.

Some registers add another complication:

- writing 1 may clear a status bit;
- reading may acknowledge an event;
- hardware may change bits between software's read and write;
- some fields may be read-only.

Therefore a correct mask is not enough. The device contract also defines whether the access sequence is safe. This distinction will matter constantly in driver and kernel work:

```
bitwise mechanism + register semantics = correct operation
```

6. Lab A — Operate a packed device register

Open the existing challenge:

```
~/masterclass/daily/challenges/day-002-register-mask-lab.c
```

It provides a fictional eight-bit register and tests for functions that:

1. set one bit;
2. clear one bit;
3. toggle one bit;
4. test one bit;
5. extract MODE;
6. replace MODE while preserving neighboring positions.

Before editing each TODO, draw three rows:

```
original
mask
result
```

Then build and run:

```
gcc -std=c17 -Wall -Wextra -Wconversion -O0 -g \  
~/masterclass/daily/challenges/day-002-register-mask-lab.c \  
-o /tmp/day-002-register-mask-lab  
  
/tmp/day-002-register-mask-lab
```

For every passing function, explain why zeros or ones in the mask preserve, force, clear, or toggle the corresponding source position. Passing tests without that explanation is useful coding evidence, but not yet mastery.

7. Fixed-width arithmetic: the answer may not fit

Pure mathematics does not run out of integer digits. Physical arithmetic units do.

A mechanical calculator has finite wheels. A relay calculator has finite relay groups. An electronic CPU has finite registers, wires, and adder stages. New switching technologies made machines dramatically faster, smaller, more reliable, and less power-hungry, but width never became free. Wider datapaths still cost circuitry, switching energy, routing space, and time.

Add one to the largest eight-bit pattern:

```
  11111111  
+ 00000001  
-----  
1 00000000
```

The full answer needs nine bits. An eight-bit destination stores `00000000`. The extra bit is a carry-out.

The adder did not make a mathematical mistake. It produced a fixed-width result. What that result means depends on the chosen interpretation.

Unsigned wrap is modular arithmetic

For an `n`-bit unsigned integer:

```
arithmetic is modulo  $2^n$ 
```

At eight bits:

```
255 + 1  $\equiv$  0    mod 256  
0 - 1   $\equiv$  255 mod 256
```

The C language defines unsigned integer arithmetic modulo the type's range. Wraparound is therefore defined behavior. It is useful for counters, hashes, ring indices, checksums, and algorithms deliberately built on modular arithmetic.

Defined does not mean automatically safe. If an allocation-size calculation wraps to a small value, later writes may exceed the allocation. The language can define the arithmetic while the program remains dangerously wrong.

Signed overflow is a different contract

Now add signed eight-bit `127 + 1`:

```
01111111  +127
+ 00000001  +1
-----
10000000  looks like -128 under two's complement
```

The low eight result bits are ordinary binary addition, but the mathematical answer `+128` is outside the signed eight-bit range.

At the CPU instruction-set level, typical arithmetic instructions produce low result bits and update status information.

At the C-language level, signed integer overflow is undefined behavior. The compiler may assume that a valid program does not perform it and optimize accordingly.

physical CPU / ISA	C abstract machine
-----	-----
produces low result bits	signed overflow has no defined result
may update status flags	optimizer may assume it never occurs

Observing wraparound in one unoptimized executable does not turn it into a portable C guarantee.

Production C should check before the operation or use an overflow-checking facility, for example:

```
int result;
if (__builtin_add_overflow(a, b, &result)) {
    /* mathematical result did not fit */
}
```

8. Carry and overflow answer different questions

One binary adder serves both unsigned and signed interpretations, so the CPU needs separate facts.

Carry asks:

Did the unsigned mathematical result need a bit beyond the available width?

Signed overflow asks:

Did the signed mathematical result fall outside the two's-complement range?

Compare two additions.

```
11111111 + 00000001 = 1 00000000
```

As unsigned: $255 + 1$ does not fit, so there is carry.

As signed: $-1 + 1 = 0$ fits, so there is no signed overflow.

Now:

```
01111111 + 00000001 = 0 10000000
```

As unsigned: $127 + 1 = 128$ fits, so there is no carry-out.

As signed: $127 + 1$ does not fit, so signed overflow occurred.

For signed addition, a useful detection rule is:

```
same-sign operands + different-sign result → signed overflow
```

At the gate level, an adder is already producing carry signals between bit positions. Logic around the highest position can derive the carry and overflow facts. Status flags preserve those facts after the result has been truncated to the register width.

This is a first glimpse of an ALU interface:

```
operand A ┐
            │ arithmetic/logic network → result bits
operand B ┘ ┘ status flags
```

The result alone is sometimes insufficient. `00000000` could be a genuine zero or the wrapped low portion of a larger answer. The flag provides additional context about the operation that produced it.

9. Changing width: preserve which interpretation?

Suppose the eight-bit pattern is:

```
11110110
```

Under unsigned interpretation it is 246. Under signed two's complement it is -10 .

To preserve the unsigned value while widening to 16 bits, **zero-extend**:

```
00000000 11110110 → 246
```

To preserve the signed value, **sign-extend** by copying the old high bit:

```
11111111 11110110 → -10
```

Why does copying ones work?

```
8-bit representation of -10:   $2^8 - 10$ 
16-bit representation of -10:  $2^{16} - 10$ 
```

The new high ones contribute exactly the place values needed to select the same negative residue at the wider modulus.

The general rule is:

```
unsigned widening → fill new high bits with 0  
signed widening   → replicate the old sign bit
```

Extension is therefore not “just adding space.” It preserves a chosen numeric interpretation while changing width.

CPUs commonly expose this distinction directly through zero-extending and sign-extending instructions. A compiler chooses between them according to the source type and operation.

10. C preview: storage width is not expression width

Consider:

```
uint8_t a = 255;  
uint8_t b = 1;  
printf("%d\n", a + b);
```

It commonly prints 256, not 0. Before addition, both operands undergo **integer promotion**. On a system where `int` can represent every `uint8_t` value, both become `int`, and the addition occurs at `int` width.

Only a later conversion narrows:

```
uint8_t result = a + b; /* 256 converted to uint8_t, yielding 0 */
```

This distinction explains why the overflow lab shows both:

```
promoted expression result  
stored eight-bit result
```

It also explains the earlier NOT example: `~x` can invert an `int`-width promoted value even when `x` is stored in eight bits.

We will derive the full promotion and conversion rules in later C work. For today, keep this preview:

```
Always distinguish the declared object's width, the promoted expression type, and the  
destination width.
```

One more language boundary matters in the lab. Converting an unsigned value that is not representable in the corresponding signed type has implementation-defined behavior in C17. On the intended GCC, two's-complement target, converting `0xf6` through `int8_t` produces the expected `-10` view. Treat that as an observed implementation choice, not a universal license to reinterpret every out-of-range conversion silently.

11. Lab B — Predict, observe, explain overflow

Open:

```
~/masterclass/daily/challenges/day-002-overflow-lab.c
```

Do not run it immediately. Read each numbered output expression and predict:

- the promoted expression value;
- the value eventually stored;
- whether the behavior is defined by C;
- whether zero extension or sign extension occurs;
- where instrumentation should report signed overflow.

Build normally:

```
gcc -std=c17 -Wall -Wextra -O0 -g \  
~/masterclass/daily/challenges/day-002-overflow-lab.c \  
-o /tmp/day-002-overflow-lab  
  
/tmp/day-002-overflow-lab
```

Then build with undefined-behavior instrumentation:

```
gcc -std=c17 -Wall -Wextra -O1 -g \  
-fsanitize=undefined -fno-sanitize-recover=undefined \  
~/masterclass/daily/challenges/day-002-overflow-lab.c \  
-o /tmp/day-002-overflow-lab-ubsan  
  
/tmp/day-002-overflow-lab-ubsan
```

Record four columns:

```
prediction | observation | C rule | hardware/CPU connection
```

In particular, explain why:

1. `u8_max + 1` can be 256 as a promoted expression;
2. storing that result in `uint8_t` yields 0;
3. zero extension and sign extension produce different high bits;
4. checked addition can report failure without first invoking signed overflow;
5. ordinary execution's apparent wrapped signed result is not a C promise;
6. UBSan stops at the intentional signed-overflow expression.

This is the complete learning loop:

```
predict → run → observe → explain → connect to the machine
```

12. Integrated investigation: one byte, several contracts

Return to:

```
11010110
```

Work through each lens without mixing rules.

Lens A: unsigned number

Use positive powers-of-two place values. State the range for eight bits and the resulting value.

Lens B: signed two's-complement number

Use both `unsigned - 256` and invert-plus-one. State why the high bit has negative significance only under this rule.

Lens C: packed register

Using the layout from section 3:

- identify READY and ERROR;
- extract MODE;
- identify IRQ_EN and START;
- write a mask to clear IRQ_EN;
- replace MODE with 5 while preserving every other bit.

Lens D: arithmetic result

Add each operand separately to the original byte:

```
00000001
00110000
10000000
```

For each addition:

1. calculate the low eight result bits;
2. record carry-out;
3. decide whether signed overflow occurred;
4. read the result as unsigned;
5. read the result as signed.

Do not say merely “it overflowed.” State **which interpretation's range did not contain the mathematical answer.**

Lens E: widened value

Write the 16-bit zero extension and 16-bit sign extension. State which eight-bit interpretation each preserves.

13. Mastery checkpoint

The tracker checkpoint is:

Given an eight-bit pattern, identify plausible interpretations and the interpretation rule.

Use a fresh pattern:

A mastery-level response should cover at least these plausible interpretations:

- **Unsigned integer:** use positive binary place values.
- **Signed integer:** specify eight-bit two's complement, then use the negative high-bit weight, subtract 2^8 , or invert-plus-one.
- **Packed flags and fields:** obtain a bit layout from the device or data format specification, then use masks and shifts.
- **Character data:** obtain a character encoding; one byte may be a complete code unit, part of a multi-byte sequence, or invalid in that context.
- **Machine instruction:** obtain the ISA, execution mode, and instruction boundary.
- **Pixel or compressed data:** obtain the format, dimensions, channel layout, and bit-order rules.

Then say the governing principle in your own words:

Bits constrain the possible interpretations, but a type, format, ISA, protocol, or device specification selects the active one.

Complete the evidence in four forms:

1. **Explain** why two's complement lets one fixed-width adder serve signed and unsigned arithmetic.
2. **Draw** the pattern, masks, field boundaries, result width, carry, and extension.
3. **Observe** both C labs, including the instrumented overflow run.
4. **Build** the TODO implementations in the register-mask lab.

You are ready to advance when you can:

- explain why signed magnitude and one's complement have two zeros;
- derive two's complement from modular arithmetic;
- derive the signed range rather than recite it;
- construct set, clear, toggle, and test masks;
- extract and replace a packed field safely;
- distinguish unsigned carry from signed overflow;
- distinguish C's defined unsigned wrap from signed-overflow undefined behavior;
- choose zero extension or sign extension for a stated reason;
- notice when integer promotions change the width of a C expression;
- and name the interpretation rule before assigning meaning to a pattern.

If one item is weak, revisit that section and create a smaller example. Do not repeat the entire unit mechanically.

14. Where this moves us: toward the ALU and CPU

We now have more than representations.

AND, OR, XOR, NOT	transform bit vectors logically
shifts	reposition bits and fields
addition	combines values and carries
status logic	reports zero, sign, carry, and overflow facts

These are central operations of an **arithmetic and logic unit**. At the physical level, transistor networks implement the gates. At the datapath level, buses bring operand patterns from registers, control signals select an operation, and result bits return to a destination register.



But an ALU is still not a CPU. We still need:

- storage so one operation can use an earlier result;
- a clock and state elements to coordinate changes;
- control logic to select ADD, AND, shift, or another operation;
- an instruction encoding that says which operation and operands to use;
- a program counter and memory path to fetch the next instruction;
- conditional behavior that can choose what happens next.

The stored-program idea makes instructions themselves bit patterns. An ISA supplies their interpretation rule, just as a device specification supplied the interpretation rule for our register:

instruction bits + ISA rule → requested CPU action

The specification is a contract. The instruction bits occupy storage. The decoder and datapath are physical circuitry. Keeping those layers distinct prevents the vague claim that meaning somehow lives inside the bits.

A conventional computer does not literally think at this level. Gates react to electrical states; registers preserve patterns; control circuitry drives sequences of transitions. Rich behavior emerges when enormous compositions of those operations implement algorithms, operating systems, applications, and learned models.

Today's operations are therefore not isolated binary tricks. They are pieces of the machinery we will combine into an ALU, then a controlled datapath, and eventually a minimal CPU.

References used selectively

- Patt and Patel, *Introduction to Computing Systems*, 2nd ed., Chapter 2, integer data types, arithmetic, logical operations, conversion, sign extension, and overflow.
- Bryant and O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed., Chapter 2, information storage, integer representations, bit-level operations, and integer arithmetic.
- Charles Petzold, *Code*, "But What About Subtraction?", for the path from complement arithmetic to binary subtraction machinery.

- Current Linux kernel headers `include/linux/bits.h` and `include/linux/bitfield.h` as examples of mask and field abstractions.
- GCC documentation for integer overflow built-ins and undefined-behavior instrumentation.

After Day 2: share the two lab results, the fresh-pattern mastery explanation, and any point where prediction differed from observation. That evidence determines whether the next unit should repair a gap or move toward logic gates, adders, and the ALU.